

Is Harness All You Need?

从 LangChain、LangGraph 到 Claude Code 的系统机制

韩成瑞

东南大学数学学院

2026 年 9 月 15 日



目录

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制
- 4 复杂任务：计划、上下文与跨会话记忆
- 5 总结与展望：从 Harness 到 Agentic RL

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制
- 4 复杂任务：计划、上下文与跨会话记忆
- 5 总结与展望：从 Harness 到 Agentic RL

研究问题：从生成到执行

中心问题

当语言模型已经能够理解任务、生成代码和选择工具时，为什么仍然需要 Agent Harness ？

- ① **能力层面**：模型如何从生成回答，转向依据环境反馈选择行动？
- ② **系统层面**：谁负责执行行动、组织状态，并构建下一轮输入？
- ③ **可靠性层面**：长任务如何保持目标、控制资源并验证结果？

本报告的中心观点

模型提供条件决策能力；Harness 将这种能力组织为受约束、可持续、可检查的任务执行过程。

统一记号与交互模型

符号	含义
q	用户目标与任务约束
π_θ	参数为 θ 的语言模型策略
h_t	已发生的交互历史
s_t	运行时维护的结构化状态
c_t	第 t 轮实际送入模型的上下文
a_t, o_{t+1}	行动请求及执行后的观察
e_t	外部环境状态

交互系统的简化描述

$$\begin{aligned}c_t &= B(q, h_t, s_t), \\a_t &\sim \pi_\theta(\cdot \mid c_t), \\(e_{t+1}, o_{t+1}) &= E(e_t, a_t), \\s_{t+1} &= U(s_t, a_t, o_{t+1}).\end{aligned}$$

B : 选择模型输入; E : 执行并观察; U : 更新运行状态。历史 h_t 随交互追加。

LLM：条件生成与执行边界

已具备的基础：语言模型根据输入上下文生成序列。

$$p_{\theta}(y \mid x) = \prod_{i=1}^{|y|} p_{\theta}(y_i \mid x, y_{<i}).$$

模型可以生成

一段代码、一份计划，或一个结构化工具请求。

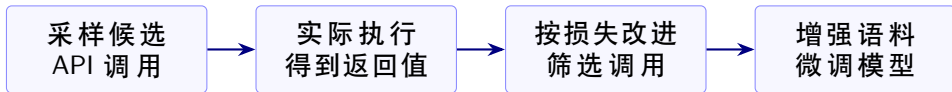
系统仍需完成

解析请求、执行工具、取得观察，并决定是否再次调用模型。

问题的变化：研究对象从输出序列 y ，扩展为由行动与观察构成的交互轨迹 τ 。

Toolformer: 工具使用的自监督学习

Schick et al. (NIPS 2023) 研究模型如何学习选择 API、调用时机和参数。



$$L_i^- - L_i^+ \geq \tau_f.$$

L_i^+ 为带调用及结果的后续 token 加权损失； L_i^- 为无调用、或只有调用无结果两种损失的较小者。

贡献与边界

训练使模型学会利用工具结果；API 的实际执行仍由外部系统承担。该方法并未给出完整长任务运行时。

Schick et al. (NeurIPS 2023)

Toolformer: 调用筛选与训练目标

设原文为 $x_{1:n}$, 在位置 i 的候选调用为 u_i , 返回值为 v_i 。

$$L_i(z) = - \sum_{j=i}^n w_{j-i} \log p_{\theta}(x_j \mid z, x_{<j}), \quad w_k \geq 0.$$

z 为附加前缀, $\text{enc}(u, v)$ 为调用及返回值的文本编码, ϵ 为空序列。

$$L_i^+ = L_i(\text{enc}(u_i, v_i)), \quad L_i^- = \min\{L_i(\epsilon), L_i(\text{enc}(u_i, \epsilon))\}.$$

筛选与学习的连接

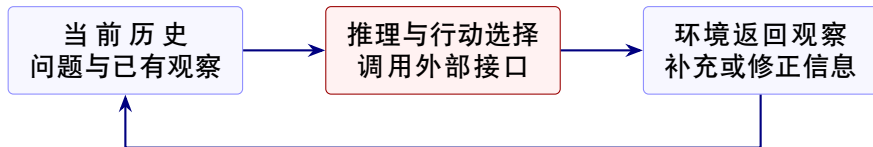
保留 $L_i^- - L_i^+ \geq \tau_f$ 的调用, 构造增强语料 $\mathcal{D}_{\text{aug}}$, 再最小化

$$\mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{x^* \sim \mathcal{D}_{\text{aug}}} \sum_j \log p_{\theta}(x_j^* \mid x_{<j}^*).$$

筛选依据为文本预测损失, 而非任务成功奖励。

Schick et al. (NeurIPS 2023)

Yao et al. (ICLR 2023) 将推理文本与任务行动交替生成，使后续决策能够利用新观察。



从静态计划到闭环决策

不是预先生成所有步骤再执行，而是让新观察影响后续行动。论文在问答、事实验证与交互任务中研究这一范式。

Yao et al. (ICLR 2023)

ReAct: 交互轨迹的形式化

将一轮中的推理文本记为 z_t ，工具行动记为 a_t ；采用按轮聚合的表达：

$$z_t \sim \pi_\theta(\cdot \mid c_t), \quad a_t \sim \pi_\theta(\cdot \mid c_t, z_t), \quad (e_{t+1}, o_{t+1}) \sim P_E(\cdot \mid e_t, a_t).$$

$$h_{t+1} = h_t \oplus (z_t, a_t, o_{t+1}), \quad c_{t+1} = B(q, h_{t+1}, s_{t+1}).$$

$\oplus$ 表示追加； P_E 为环境转移与观察分布。确定性环境是其特例。

关键依赖：环境反馈进入下一轮条件分布

在给定初始状态与 Harness 下，长度为 T 的轨迹概率分解为

$$p_\theta(\tau \mid q, e_0, \mathcal{H}) = \prod_{t=0}^{T-1} \pi_\theta(z_t \mid c_t) \pi_\theta(a_t \mid c_t, z_t) P_E(e_{t+1}, o_{t+1} \mid e_t, a_t).$$

原论文允许灵活交替推理与行动；这里按轮聚合以说明概率依赖，不要求每步都有推理文本。

Yao et al. (ICLR 2023)

ReAct: 任务表现与适用范围

原论文 Table 1: PaLM-540B, few-shot prompting; 以下为部分结果 (%)。

方法	HotpotQA (EM)	FEVER (Accuracy)
Standard	28.7	57.1
CoT	29.4	56.3
Act	25.7	58.9
ReAct	27.4	60.9
ReAct $\rightarrow$ CoT-SC	35.1	62.0

如何解释这组证据

ReAct 在这两项任务上均优于 Act；但在 HotpotQA 上低于 CoT。外部信息质量、检索过程和推理策略会共同影响结果。

Yao et al. (ICLR 2023)

维度	Toolformer	ReAct
核心问题	如何学会调用及利用工具？	如何在行动中持续更新决策？
主要方法	构造带 API 的语料并微调	推理、行动与观察交替组织
主要位置	模型能力与训练数据	推理时的交互策略
共同依赖	外部接口执行，以及返回值进入后续模型输入	

尚未解决的问题

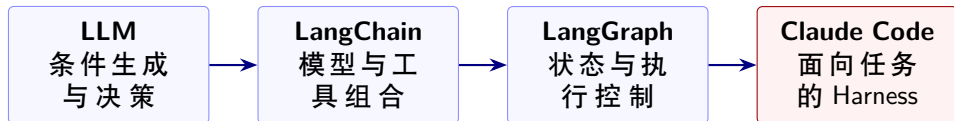
即使模型已经会选择行动，多工具组合、状态更新、执行恢复与上下文组织仍需系统实现。

Yao et al. (ICLR 2023) Schick et al. (NeurIPS 2023)

目录

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制
- 4 复杂任务：计划、上下文与跨会话记忆
- 5 总结与展望：从 Harness 到 Agentic RL

从模型到任务系统



- ① 从模型调用走向可组合的应用组件。
- ② 从组件组合走向可显式管理的状态与控制流。
- ③ 从运行时能力走向具体任务环境中的机制配置。

脉络的含义

箭头仅表示本报告的问题递进。

[LangChain, *LangChain Overview*.](#) [LangChain, *LangGraph Overview*.](#) [Anthropic, *How Claude Code Works*.](#)

LangChain：接口抽象与组件组合

问题：不同模型、工具与消息格式如何协同工作？

核心抽象

模型接口、消息对象、工具描述与调用、Agent 循环的高层封装。

组合原则

开发者以组件及其契约组织应用，无需为每种组合重写全部连接逻辑。

$$x_0 = x, \quad x_i = f_i(x_{i-1}) \ (i = 1, \dots, n), \quad y = x_n.$$

x 、 y 分别为应用输入与输出； f_i 为第 i 个组件， x_i 为其中间结果。相邻组件的输出与输入须兼容，例如提示构建、模型调用与结果解析。线性复合仅说明组合原则；工具调用循环还需要控制流。

控制流需求

当执行存在循环、分支、中断与重试时，控制流和状态应该如何表达？

LangChain, *LangChain Overview*.

LangGraph: 状态更新与路由

以单个活动节点的顺序执行说明：图 $G = (V, E)$ 中， V 为计算节点集合， E 为允许的转移关系。

$$\Delta s_t = f_{v_t}(s_t), \quad s_{t+1} = R(s_t, \Delta s_t), \quad v_{t+1} = \rho_{v_t}(s_{t+1}).$$

符号	含义
s_t, v_t	第 t 步的共享状态与当前活动节点， $v_t \in V$ 。
$f_v, \Delta s_t$	节点 v 的计算与它返回的部分状态更新；不必重写全部状态。
R	按状态字段应用合并规则，例如追加消息或覆盖计数值。
ρ_v	节点 v 的后继选择规则；可以固定转移，也可依据状态条件路由至结束。

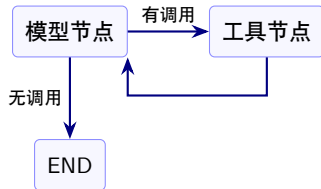
更新与调度的分工

节点产生更新，合并规则构造新状态，路由决定后继节点。实际运行还支持并行执行；上式为顺序情形的抽象。

LangGraph: 工具循环示例

设共享状态 $s = (M, k)$: M 为消息序列, k 为已执行工具批次的计数。

- ① 模型节点返回消息更新 $\Delta M = [r]$, 其中 r 为模型回复。
- ② 合并规则追加消息: $M' = M \oplus [r]$; 未更新的 k 保持原值。
- ③ 路由检查 r : 有工具请求则进入工具节点, 否则结束。
- ④ 工具节点回传结果消息, 并将计数更新为 $k + 1$; 随后返回模型节点。



状态字段具有不同语义

消息采用追加规则; 计数采用覆盖规则。⊕ 表示序列拼接, END 表示图执行终点。

Workflow 与 Agent：控制权分配

比较维度	Workflow	Agent
下一步主要由谁决定	预定义规则与代码路径	模型结合新观察选择
可以包含 LLM 吗	可以	可以
可以使用图吗	可以	可以
较适合的情形	步骤和判定条件较明确	路径需要随环境信息调整

两种控制可以共存

例如：模型决定读取哪个文件；代码检查参数、执行工具，并约束最多运行多少轮。

Anthropic, Building Effective Agents (2024).

任务执行的系统约束

引导情境：在代码仓库中修复一个缺陷，并确认修改满足需求。

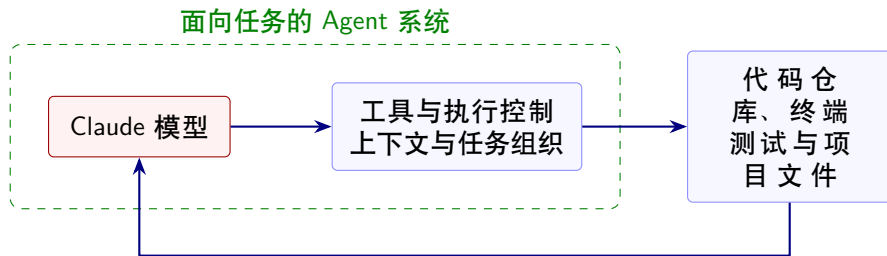
- ① 模型能够提出编辑，图运行时能够调度执行：哪些行动应获准，失败如何返回？
- ② 修改跨越多个文件与回合：目标、进度与工作现场如何保持一致？
- ③ 调查产生大量日志与规则：下一轮保留什么，何时读取其他信息？
- ④ 模型提出结束：什么证据足以支持交付？

从运行时到任务组织

运行时提供状态与控制流的表达能力；具体任务还需要执行、信息与验证的组织规则。
Claude Code 提供一个观察这些规则如何协同工作的实例。

Claude Code: 面向任务的系统组织

官方说明: Claude Code 在 Claude 模型外围提供工具、上下文管理与执行环境。



工作方式: 收集上下文、采取行动、验证结果，依据观察继续调整。三者在实际循环中可以交错。

Anthropic, How Claude Code Works.

工作定义

本报告将 Harness 理解为围绕模型构建的执行与信息组织机制：规定模型能采取什么行动、看到什么信息，以及系统如何维持和检查任务过程。

分析层次	主要关注	代表入口
模型	给定输入如何生成决策	LLM
框架	如何组合组件	LangChain
运行时	如何执行状态与控制流	LangGraph
Harness	如何配置任务行为及边界	Claude Code

Harness: 任务约束与机制选择

任务约束	系统职责	机制实例
行动须影响真实环境 操作存在副作用与成本	执行请求并返回可用反馈 限定行动范围及执行预算	工具接口、执行循环 权限、隔离、终止条件
任务跨越多个步骤 知识与历史超出输入 预算	维护进度及状态连续性 构建与当前决策相关的输入	Todo、任务文件、检查点 Skill、Subagent、Compact
产物需要满足目标	依据环境证据判定完成	测试、验证与验收

机制选择原则

必要性来自任务约束。同一职责可由不同机制实现；后文结合 learn-claude-code，依次讨论行动基础、复杂任务与跨会话知识，再引出完成判据。

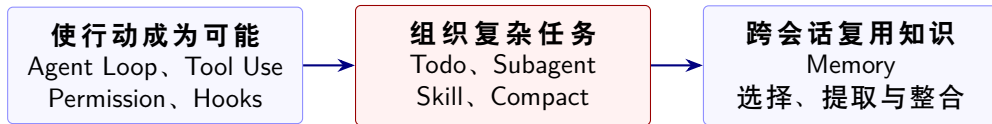
shareAI-lab, [learn-claude-code \(GitHub\)](#).

目录

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制**
- 4 复杂任务：计划、上下文与跨会话记忆
- 5 总结与展望：从 Harness 到 Agentic RL

Harness 机制的递进结构

以代码任务为例：读取项目、定位问题、完成修改，并在后续任务中复用相关知识。



- ① 行动基础：模型提出请求，系统将请求转为受控执行与反馈。
- ② 复杂任务：显式维护目标，并组织局部调查、外部知识与增长的历史。
- ③ 跨会话知识：从交互中保留可复用信息，在相关任务中召回。

分析原则

沿 learn-claude-code 的基础学习路线展开；每项机制对应一种任务约束，具体实现可替换。

shareAI-lab, [learn-claude-code](#) (GitHub).

Agent Loop: 请求、执行与反馈

模型选择行动；循环保存调用与结果，使环境观察进入下一轮决策。

```
while within_budget():
    reply = model(messages, tools)
    messages.append(reply)
    calls = extract_tool_calls(reply)
    if not calls:
        return reply
    for call in calls:
        result = execute(call)
        out = tool_result(call.id, result)
        messages.append(out)
```

- ① 请求携带工具名、参数与调用标识。
- ② 运行时调用对应处理函数。
- ③ 结果按标识关联原请求。
- ④ 模型据新观察继续行动。

核心协议

一个 Bash 工具即可构成交互循环；增加工具时，保留请求、执行、回传这一控制结构。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

Tool Use: 动作定义与统一分发

从单一 Bash 入口扩展到读取、搜索与编辑：关键是为模型和运行时提供一致的工具契约。

$$a_t = (n_t, x_t), \quad o_{t+1} = F_{n_t}(x_t; e_t).$$

n_t 为工具名, x_t 为参数, F_n 为对应处理函数; e_t 为环境状态, o_{t+1} 为执行反馈。

接口组成	对模型的作用	运行时职责
名称、描述与参数模式	说明何时调用、如何填写参数	校验并分发到处理函数
操作语义	确定读取范围或修改方式	执行指定操作, 捕获异常
结果与错误格式	支持理解现场、修正下一步	返回输出、错误及调用标识

从协议到接口设计

工具注册解决能力扩展; 动作粒度与反馈表示决定模型如何使用这些能力。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

SWE-agent: 接口设计的实验证据

研究问题： 固定基础模型，改变行动接口和观察组织，任务解决率是否随之变化？

SWE-bench Lite, 300 个任务, GPT-4 Turbo; 解决率 (%)。

系统配置	Resolved
Shell-only agent	11.0
SWE-agent	18.0

SWE-agent 消融	Resolved
默认配置	18.0
文件查看: 完整文件	12.7
上下文: 完整历史	15.0
编辑: 不带 linting	15.0

接口作为系统变量

左侧比较整体配置；右侧考察观察范围、历史组织与编辑反馈。在该设置下，接口配置影响任务表现，整体提升不能归因于某一个工具。

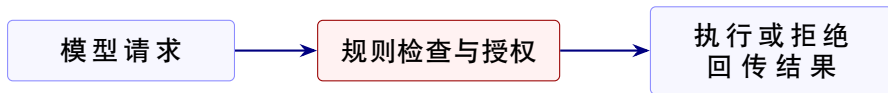
Yang et al. (NeurIPS 2024)

Permission: 行动的授权边界

工具描述回答“能提出什么请求”；权限规则进一步决定“这次请求是否执行”。

$$P(a_t, s_t) \in \{\text{allow}, \text{ask}, \text{deny}\}.$$

P 为授权规则，依据行动及运行状态决定直接允许、请求确认或拒绝。



请求层：权限检查

教学实现检查命令及访问路径；拒绝信息进入工具反馈，使模型能调整行动。

资源层：环境隔离

操作系统或容器限制进程实际可访问的资源。应用层命令检查不能等同于沙箱隔离。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

Hooks：执行生命周期的扩展点

权限、日志与结果检查需要在特定时点执行。Hooks 将这些规则注册到稳定的循环事件上。

事件	触发时点	教学实现中的作用
UserPromptSubmit	用户输入后	触发输入侧回调，例如记录工作目录
PreToolUse	工具执行前	执行权限检查；可阻止本次工具调用
PostToolUse	工具执行后	记录或检查输出，例如提示输出过大
Stop	模型提出结束时	回调可允许返回，或加入反馈继续循环

机制分工

Permission 定义授权规则；Hooks 定义规则介入的时点。具体回调决定实际行为。

到此，行动可以被执行、约束和扩展；接下来需要组织多步目标及其产生的信息。

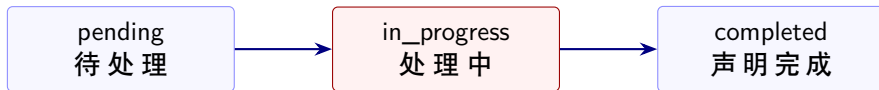
[shareAI-lab](#), [learn-claude-code](#) (GitHub).

目录

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制
- 4 复杂任务：计划、上下文与跨会话记忆**
- 5 总结与展望：从 Harness 到 Agentic RL

Todo: 多步目标的显式维护

行动循环可以持续执行，但多步任务还需要显式保留“待做什么、正在做什么”。



$$s_{t+1} = U(s_t, a_t, o_{t+1}), \quad c_{t+1} = B(q, h_{t+1}, s_{t+1}).$$

s_t 为任务记录； U 表示更新， B 将记录纳入模型输入。图示为典型推进顺序。

教学实现中的反馈机制

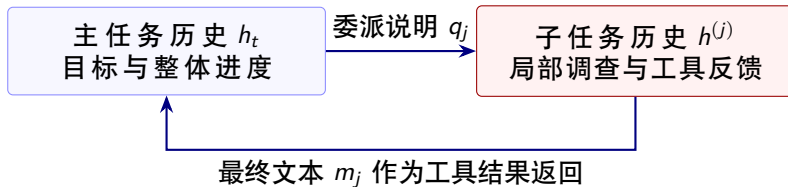
模型通过工具提交清单；运行时校验内容与状态并回传清单。若多轮未更新，则加入提醒，使计划重新进入模型可见信息。

记录须依据文件与测试事实校正；completed 本身不构成验收证据。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

Subagent: 局部任务与独立历史

有了计划，局部调查仍可能产生大量中间记录。子任务可复用同一循环，在独立消息历史中完成。



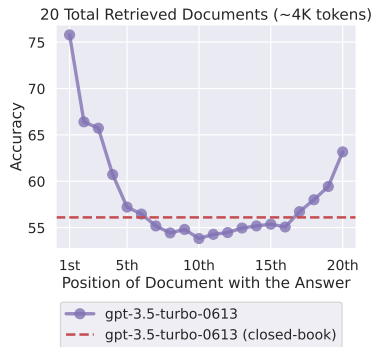
$$h_0^{(j)} = [q_j], \quad m_j = \text{Run}(h_0^{(j)}), \quad h_{t+1} = h_t \oplus \text{result}(m_j).$$

j 为子任务编号，Run 表示子循环， $\oplus$ 表示追加；主历史保留对应委派请求。

隔离的对象

教学实现共享工作目录，但不把子任务完整历史带回主任务。消息隔离不等于文件隔离；返回结论的证据质量仍取决于任务说明与执行结果。

上下文约束：容量与信息利用



子任务缩减主历史的增长，但每个循环仍受输入预算限制：

$$c_t = B(q, h_t, s_t), \quad |c_t|_{\text{tok}} \leq K.$$

K 为输入 token 预算， $|\cdot|_{\text{tok}}$ 为 token 数。

容量之外还有利用问题：左图中，相关文档位于输入中部时准确率下降。

实验设置：20 篇文档、约 4K tokens；
GPT-3.5-turbo-0613。

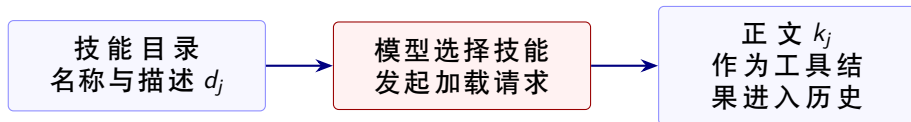
进一步的设计问题

外部知识应在何时进入输入？既有历史应以何种形式保留？分别引出 Skill 与 Compact。

Liu et al. (TACL 2024)

Skill: 知识目录与按需加载

项目规范与领域流程并非每轮都相关。先提供可检索的描述，再按任务加载正文。



$$c_t = \text{pack}(q, s_t, h_t, d_{1:m}, \{k_j : j \in J_t\}).$$

m 为技能总数， J_t 为已加载技能索引；`pack` 表示在预算内组织输入，避免重复装入已在历史中的内容。

知识供给与工具执行

Skill 提供操作知识，例如项目测试流程；工具负责实际执行测试。加载知识不改变模型参数，也不自动完成所描述的操作。

[shareAI-lab](#), [learn-claude-code](#) (GitHub). Anthropic, *Extend Claude with Skills*.

Compact: 分层控制历史规模

按需加载控制新增知识；已经积累的工具输出与交互历史，则需要逐层缩减。

处理层次	主要操作	留在上下文中的信息
工具结果预算	将过大的输出写入外部文件	预览与文件引用
旧消息归档	保留首尾，将较早的中间段归档	近期交互与归档标记
旧结果缩减	将已使用的较早工具结果替换为引用	调用关联与读取路径
历史摘要	上述处理后仍超限时，再生成摘要	当前请求与继续工作所需信息

分层处理的意义

先缩减可回读的冗长结果，再进行语义摘要；保留工具请求与结果的配对关系，避免压缩破坏交互协议。

这些操作服务于当前会话的续行。跨会话需要复用的知识，还需单独选择与存储。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

Memory: 跨会话知识的选择与复用

新会话没有旧消息历史，但项目事实、长期偏好或有效经验可能仍有价值。

环节	机制	需要判断的问题
提取	从已完成的交互中形成候选记录	是否具有跨任务价值？
存储	保存记录正文，并建立简短索引	如何定位、更新与追溯？
召回	按当前请求选择，再加载相关正文	本轮是否相关、是否仍有效？
整合	合并重复记录，处理冲突与过时内容	是否应修订或删除旧知识？

$$\mathcal{M}^+ = \text{Consolidate}(\mathcal{M} \cup \text{Extract}(h)), \quad r_q = \text{Recall}(q, \mathcal{M}^+).$$

$\mathcal{M}$ 为持久知识库， h 为会话历史， r_q 为当前请求 q 召回的记录；上式概括知识更新与读取过程。

持久化的选择性

保留长期适用的信息；临时任务指令不自动转为长期规则。召回记录作为背景知识，其有效性须结合当前请求和环境核对。

机制协同：从行动到知识积累

机制	直接管理的对象	主要时间范围
Loop / Tool Use	行动请求与环境反馈	每次模型调用
Permission / Hooks	执行规则及其介入时点	每次操作的生命周期
Todo	当前目标与推进状态	当前任务
Subagent	局部任务的交互历史	子任务执行期间
Skill	预先组织的知识与流程	需要时加载
Compact	活跃会话的历史表示	当前会话续行
Memory	从交互中提取的可复用知识	跨会话

从机制到任务结果

这些机制分别组织行动、目标与信息，共同服务于任务执行。下一章讨论如何依据产物判定完成，以及如何利用交互轨迹学习策略。

[shareAI-lab](#), [learn-claude-code](#) (GitHub).

目录

- 1 问题起点：从语言生成到环境交互
- 2 系统脉络：从组件抽象到 Agent Harness
- 3 行动基础：循环、工具与执行控制
- 4 复杂任务：计划、上下文与跨会话记忆
- 5 总结与展望：从 Harness 到 Agentic RL

任务完成：停止信号与验收证据

行动、状态和上下文都已组织起来；交付仍需要目标与外部产物之间的检验关系。

$$\hat{Y} = V(q, e_T, \mathcal{Z}_T), \quad \{a_T = \text{stop}\} \not\Rightarrow \{\hat{Y} = 1\}.$$

$\mathcal{Z}_T$ 为测试与检查记录， V 为验收规则， $\hat{Y}$ 为其判断。

停止信号与验收证据承担不同职责

模型停止或 Todo 完成是过程信号；测试与产物检查在其覆盖范围内支持完成。验收规则自身也可能遗漏需求，因此评估训练奖励时仍需独立验证。

自然延伸：能否从这些交互中学习更好的策略？

此前固定模型参数、设计 Harness；下一步可利用环境反馈更新策略，使模型学会更有效地调用、观察与修正。

Agentic RL: 交互策略优化

令 $\tau \sim p_{\theta, \mathcal{H}}(\cdot | q)$, r_t 为环境或评估器给出的奖励:

$$J(\theta; \mathcal{H}) = \mathbb{E}_{q \sim \mathcal{D}, \tau \sim p_{\theta, \mathcal{H}}} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \quad 0 < \gamma \leq 1.$$

在固定环境及 Harness、奖励无显式参数依赖时, 策略梯度可写为

$$\nabla_{\theta} J = \mathbb{E} \left[\sum_t \gamma^t \nabla_{\theta} \log \pi_{\theta}(b_t | c_t) (G_t - \beta(c_t)) \right], \quad G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k.$$

b_t 聚合模型生成的推理与行动, β 为基线; 工具返回的观察不作为模型生成 token 求对数概率。

与前文的关系

Toolformer 用文本预测损失学习调用; ReAct 组织交互; Agentic RL 进一步用任务回报优化交互策略。Harness 决定采样时的工具、上下文与终止规则, 从而影响训练轨迹分布。

研究展望：策略与 Harness 的交互效应

已有切入点：RAGEN 研究多轮 RL 的训练稳定性与轨迹采样；Agent Lightning 将执行与训练解耦并处理信用分配。

研究问题	对照设计	观察指标
训练是否改变机制收益？	同一起点，比较训练前后在两种 Harness 下的表现	独立验收成功率、调用成本
压缩如何影响策略学习？	固定任务与训练预算，改变压缩规则	学习曲线、约束遗漏、回读次数
奖励是否替代了真实目标？	训练用可验证奖励，评估用独立保留测试	训练奖励与保留测试的差距

$$\Delta_{\text{int}} = [J(\theta_1; \mathcal{H}_1) - J(\theta_1; \mathcal{H}_0)] - [J(\theta_0; \mathcal{H}_1) - J(\theta_0; \mathcal{H}_0)].$$

θ_0, θ_1 为训练前后策略；四项在相同评估任务、奖励口径与推理预算下测量。该差分刻画机制收益是否随训练改变；用多次训练和重复评估报告不确定性。

Wang et al. (arXiv 2025) Luo et al. (arXiv 2025)

结论：模型、机制与任务表现

- ① **模型与交互**：工具使用能力进入策略，环境观察进入后续决策；分析对象扩展为完整轨迹。
- ② **框架与 Harness**：组件和运行时提供组织能力；任务中的执行、状态、信息与验收问题决定具体机制。
- ③ **从设计到学习**：Harness 改变交互分布，Agentic RL 在该分布上更新策略；两者的配合可通过受控实验研究。

贯穿全文的研究对象

任务与环境 $\longleftrightarrow$ Harness $\longleftrightarrow$ π_θ , 以独立验收质量和资源成本共同评价。

参考文献： 研究论文 (1/2)

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. *ReAct: Synergizing Reasoning and Acting in Language Models*. The Eleventh International Conference on Learning Representations (ICLR), 2023.
OpenReview: WE_vluYUL-X
- Schick, T., Dwivedi-Yu, J., Dessì, R., et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. *Advances in Neural Information Processing Systems*, 36, 2023, pp. 68539–68551. NeurIPS 2023 Proceedings
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. *Advances in Neural Information Processing Systems*, 37, 2024. NeurIPS 2024 Proceedings

参考文献： 研究论文 (2/2)

- Liu, N. F., Lin, K., Hewitt, J., et al. *Lost in the Middle: How Language Models Use Long Contexts*. TACL, 12:157–173, 2024.
<https://aclanthology.org/2024.tacl-1.9/>
- Wang, Z., Wang, K., Wang, Q., et al. *RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning*. arXiv preprint arXiv:2504.20073, 2025.
<https://arxiv.org/abs/2504.20073>
- Luo, X., Zhang, Y., He, Z., et al. *Agent Lightning: Train ANY AI Agents with Reinforcement Learning*. arXiv preprint arXiv:2508.03680, 2025.
<https://arxiv.org/abs/2508.03680>

参考资料：框架与产品文档

- LangChain. *LangChain Overview*.
<https://docs.langchain.com/oss/python/langchain/overview>
- LangChain. *LangGraph Overview*.
<https://docs.langchain.com/oss/python/langgraph/overview>
- LangChain. *Graph API Overview*.
<https://docs.langchain.com/oss/python/langgraph/graph-api>
- Anthropic. *How Claude Code Works*.
<https://code.claude.com/docs/en/how-claude-code-works>
- Anthropic. *Extend Claude with Skills*.
<https://code.claude.com/docs/en/skills>
- Anthropic. *Create Custom Subagents*.
<https://code.claude.com/docs/en/sub-agents>

参考资料：实践文章与教学项目

- Anthropic. *Building Effective Agents*. 2024.
<https://www.anthropic.com/engineering/building-effective-agents>
- Anthropic. *Effective Context Engineering for AI Agents*. 2025.
<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- Anthropic. *Effective Harnesses for Long-Running Agents*. 2025.
<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>
- shareAI-lab. *learn-claude-code*. GitHub repository.
<https://github.com/shareAI-lab/learn-claude-code>